marola-bench: the design proposal, in pictures

A preprint of the system that makes the bench: points, actors, a Pekko cluster on GCP

Matheus Hoffmann

2026-10-14





Abstract

This preprint draws the system that the marola-bench proposal describes in words. The bench is the set of score series: the scores of any forecaster against the observations of a point, in order of time. A person adds each observation point through a CRUD API. One actor for each point asks the forecasters, gets the observation and registers the score. The actors live in an Apache Pekko cluster on GCP, defined as code in Besom. The cluster is cheap because it is reliable: it can use virtual machines that GCP may take back. Six figures show the bench, the system, the life of an actor, the loss of a node, the seven hard rules of the owner, and how a change is made. Every box that the design has not decided is dashed. The preprint designs nothing, and it reports no result.

Cite this as. Hoffmann, M. (2026). *marola-bench: a proposal for a WeatherBench for Brazil*. Zenodo. https://doi.org/10.5281/zenodo.23352633. All rights reserved: read it and cite it; do not copy it or share it without the author's acknowledgement.

What this is. A preprint that draws section 4 and the hard rules of the marola-bench paper (`paper.md`). It adds no design. A solid box is a hard rule of the owner, or a part of the paper's section 4. A dashed box is open: the next MIP decides it. If this preprint and the paper disagree, the paper is right. If either disagrees with MIP-0091, the MIP is right until the next MIP (Hoffmann 2026).

How to read the marks. **(v)** means that the claim was checked against its source; `REFERENCES.md` quotes the passage. **(MIP §n)** means that the claim is the design's. **(paper §n)** means that the claim is the paper's own proposal, from that section. ⚠ means not checked by anyone yet.

1. The bench

"The bench are the score series performed by any Forecaster, Registered by an Actor." (the owner, 13 October 2026)

- An **observation point** has an instrument, an averaging, a quality flag and a licence tier **(MIP R4)**. A person adds it by hand **(paper §4.2)**.
- A **forecaster** is a provider's model today, and marola's own forecaster later. Each one is scored by the same rule **(paper §4.1)**.

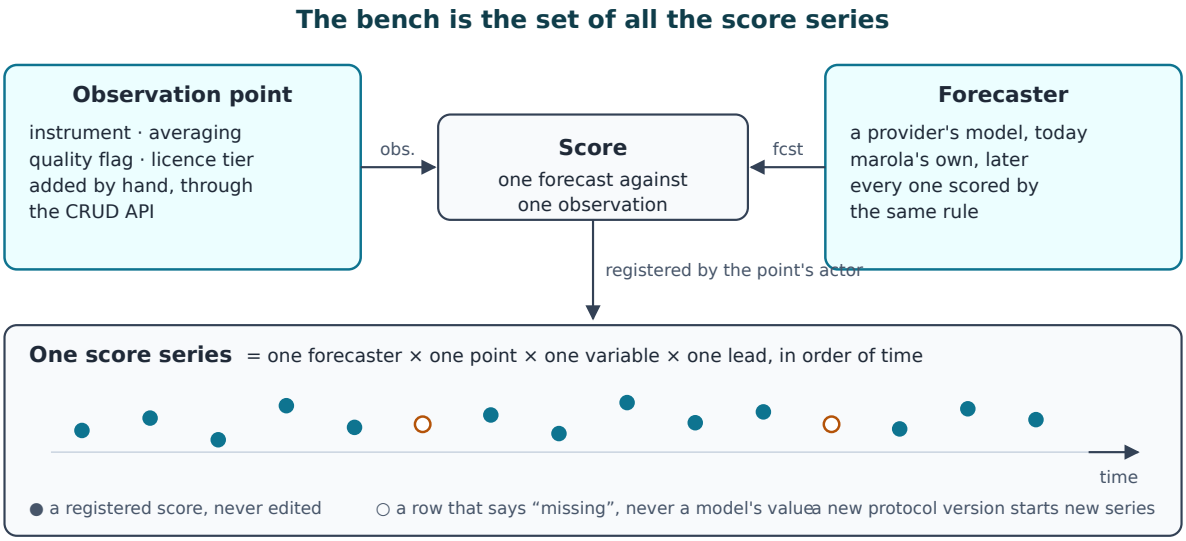


Figure 1: The bench is the set of all the score series. A score is one forecast against one observation. A series is one forecaster, one point, one variable and one lead, in order of time.

- A score that is registered is never edited. A new version of the protocol starts new series **(MIP R2)**. A missing observation is a row that says "missing", never a model's value **(MIP R4)**.

2. The system

- The CRUD API is specified in Apache Pekko and Scala **(paper §4.2)**. Pekko HTTP is "a full server- and client-side HTTP stack on top of pekko-actor and pekko-stream" (The Apache Software Foundation, n.d.d) **(v)**.
- One actor for each point. An actor with nothing to do is stopped and started again when it is needed: Pekko calls this passivation, and by default an idle entity is passivated after 2 minutes (The Apache Software Foundation, n.d.a) **(v)**.
- The cluster runs on GCP (hard rule 2), in Scala 3 and Pekko (hard rule 1). Its infrastructure is code in Besom (hard rule 3) (VirtusLab, n.d.) **(v)**.
- Open: where the events are stored, which GCP products run the nodes, how many nodes, and the budget in money **(paper §4.4)** ⚠.

3. The life of a point's actor

The actor monitors its point, asks the providers, later asks marola's forecaster, and registers each score **(paper §4.3)**. The order of steps 3 and 4 in the figure is a drawing, not a decision. Two of the paper's proposals for the next MIP show in it **(paper §4.4)**:

- **State is events.** The actor stores what happened and builds its state from that. Pekko recovers such an actor "by replaying the stored events", also when it is "migrated within a cluster" (The Apache Software Foundation, n.d.b) **(v)**.

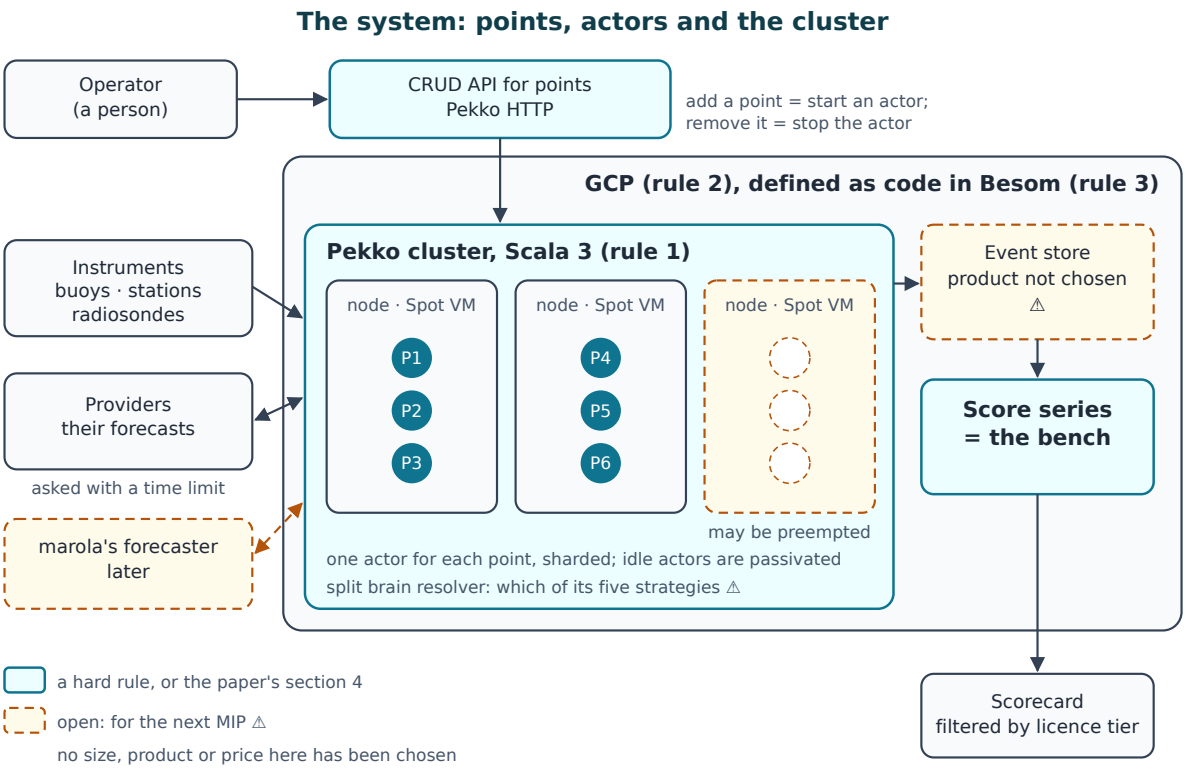


Figure 2: The system: a CRUD API starts and stops one actor for each point, inside a Pekko cluster on GCP. A dashed box is open.

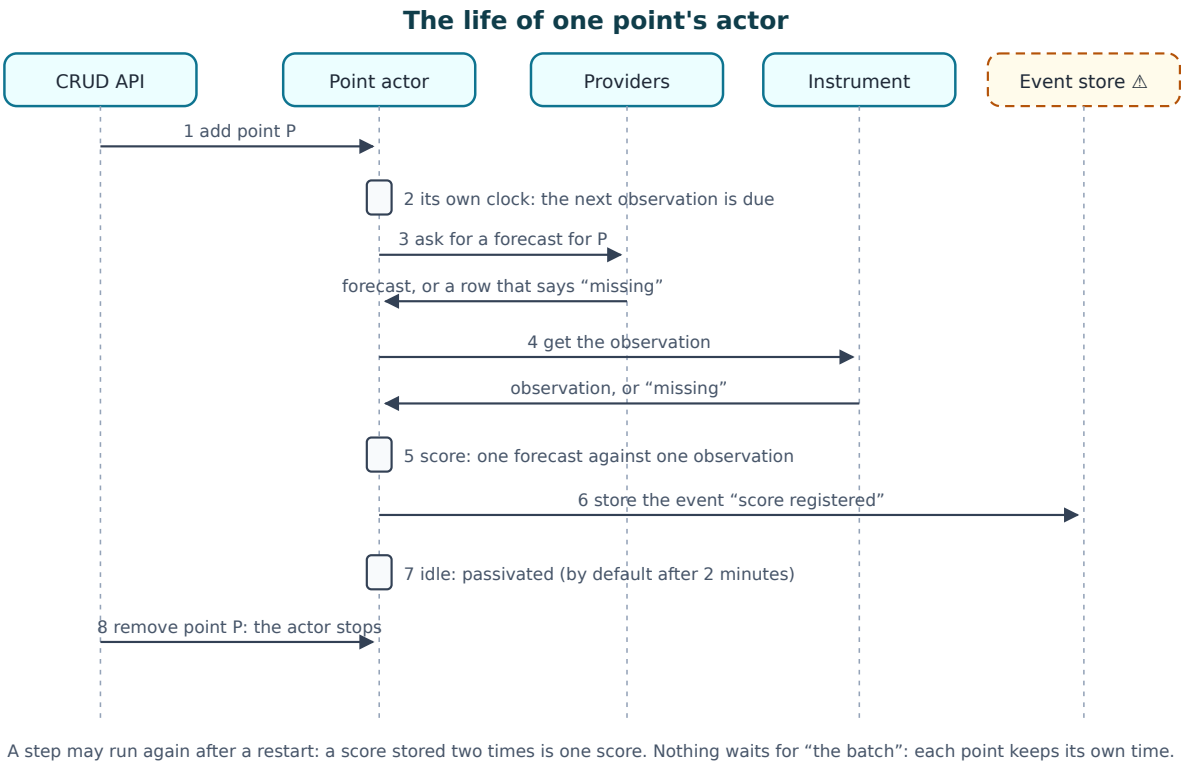


Figure 3: The life of one point’s actor, from the request that adds its point to the one that removes it.

- **Every write can be repeated.** After a restart the actor may do its last step again. A score registered two times is one score.

4. Reliability is what makes it cheap

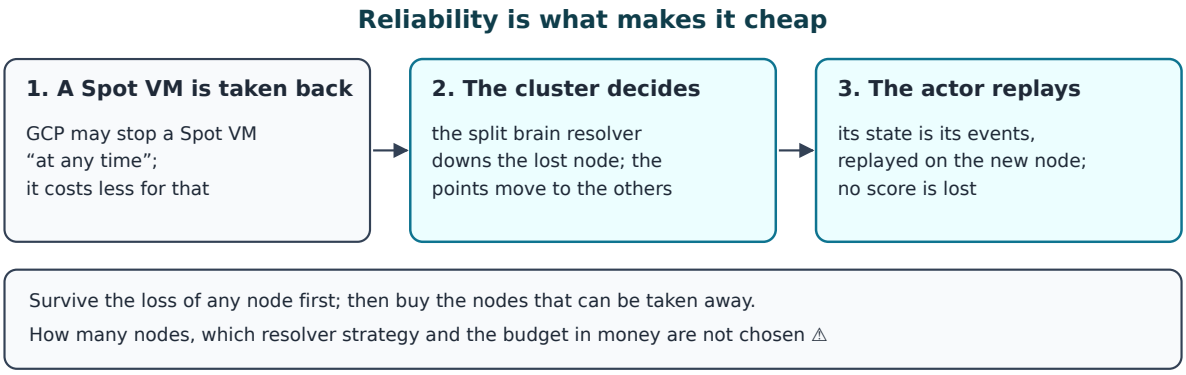


Figure 4: The loss of one node. A Spot VM is taken back, the cluster decides, and the actor replays its events on another node.

On GCP, “Compute Engine might preemptively stop or delete (preempt) Spot VMs to reclaim the capacity at any time”, and Spot VMs have “significant discounts” (Google Cloud, n.d.) **(v)**. A cluster that survives the loss of a node can use them. So the two halves of hard rule 5 are one piece of work: first survive the loss of any node, then buy the nodes that can be taken away **(paper §4.4)**.

When nodes cannot see each other, both sides must “take the same decision about which part will keep running”. Pekko’s Split Brain Resolver has five strategies, and keep majority is the default (The Apache Software Foundation, n.d.c) **(v)**. Which one marola-bench uses is open ⚠.

5. The seven hard rules

The rules are in the paper, with what each one meets **(paper, “Hard rules”)**. Rule 7 is of 14 October 2026: Ponytail 5 is the coding assistant, a Claude Code plugin (Gebert, n.d.) **(v)**. It agrees with MIP-0091, which was written with Claude Code and names no plugin **(MIP header)**. Rule 3 is drawn as “adds”, not “changes”: the design provisions nothing, so it has no infrastructure as code to change **(MIP R17)**.

6. How a change is made

- A design change is written in a MIP in `marola-dev/marola` first. A paper gives an account of a design; it does not make one (this repository’s `AGENTS.md`, rule 6) **(v)**.
- Ponytail’s skill takes “the smallest change that fully solves the task” **(v)**. Its skill file does not mention a specification **(v)**. So it does not fill rule 2’s SDD framework, and where that framework sits in the chain is open ⚠. This is the writer’s reading.
- The owner merges. An agent never merges and never releases (`AGENTS.md`, rule 7) **(v)**.

The seven hard rules, against the accepted design (MIP-0091)

Rule		What it meets in MIP-0091	
1	Apache Pekko, Scala 3	Scala 3 agrees; Kyo becomes Pekko	changes
2	GCP, Claude Code, an SDD framework	zero spend meets a paid cloud; SDD unnamed	changes
3	Besom	the design provisions nothing, so it has no IaC	adds
4	lives in marola-research/marola-bench	no new repository agrees; the home moves	changes
5	a cheap and reliable cluster	same aims; an always-on cluster on a paid cloud	changes
6	all rights reserved	the design wants CC BY 4.0 scorecards	changes
7	Ponytail 5, a Claude Code plugin	written with Claude Code; names no plugin	agrees

The rules are the owner's later word, so they win; how each change is settled is for the next MIP.

Figure 5: The seven hard rules of the owner, and what each one meets in MIP-0091.

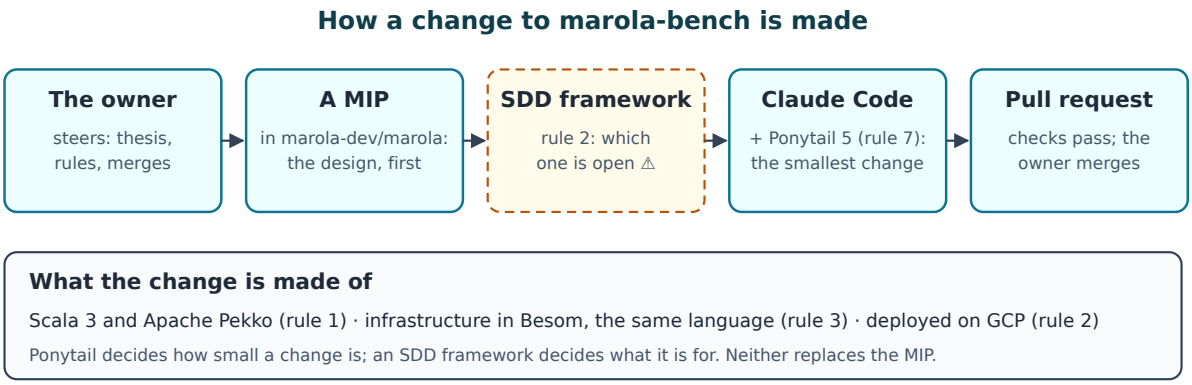


Figure 6: How a change to marola-bench is made: the owner steers, a MIP comes first, and Claude Code with Ponytail writes the change.

7. What is not known

- Every dashed box: the event store, the GCP products, the number of nodes, the strategy of the Split Brain Resolver, the budget in money, and the SDD framework ⚠.
- Whether Pekko and the Kyo module of the accepted design live together, or one replaces the other **(paper, "Hard rules")**.
- How the actors and the four-hour cycle of today meet **(MIP §5.2)**.
- What Ponytail changes in this work. Its page reports numbers from its author's benchmark, and nobody has measured them on marola-bench, which has no code yet ⚠.

8. Where this comes from

The paper `paper.md`, its section 4 and its hard rules, read on 14 October 2026. The design, MIP-0091 at commit `cf7bd4f` (Hoffmann 2026). The figures were drawn for this preprint, as SVG files in `figures/`; they copy no other figure.

References

How each entry was checked is in `REFERENCES.md`.

Gebert, Dietrich. n.d. "Ponytail: Lazy Senior Dev Mode." https://github.com/DietrichGebert/ponytail/tree/2961b52fc1fe6a5a87c7a959a79512e5c2f3ffe5.

Google Cloud. n.d. "Compute Engine Documentation: Spot VMs." https://cloud.google.com/compute/docs/instances/spot.

Hoffmann, Matheus. 2026. "MIP-0091: The Marola Forecaster's Architecture: The Evaluation Goes First, No New Repo, and the Design Requirements from Read, Evaluate, Adopt on Operational WeatherBench." https://github.com/marola-dev/marola/blob/cf7bd4f/docs/MIPs/MIP-0091-marola-forecaster-architecture.md.

The Apache Software Foundation. n.d.a. "Apache Pekko Documentation: Cluster Sharding." https://pekko.apache.org/docs/pekko/current/typed/cluster-sharding.html.

———. n.d.b. "Apache Pekko Documentation: Event Sourcing." https://pekko.apache.org/docs/pekko/current/typed/persistence.html.

———. n.d.c. "Apache Pekko Documentation: Split Brain Resolver." https://pekko.apache.org/docs/pekko/current/split-brain-resolver.html.

———. n.d.d. "Apache Pekko HTTP Documentation: Introduction." https://pekko.apache.org/docs/pekko-http/current/introduction.html.

VirtusLab. n.d. "Besom: A Pulumi SDK for Scala." https://github.com/VirtusLab/besom.